Complete presentation and marked-geometry programs

L095 recovery

These three files are required exact computations. The marking construction is unchanged from the complete L084 census packet, and is counted once as a reused file. The other two establish the presented-group upper bound and the actual geometric interfaces. The full census programs, data and tables remain included separately; this appendix does not replace or shorten them. Runtime implementations and named group data are retained and pinned by the runner. No Lean verification is claimed.

Presented-group upper bound

```
# Exact upper bound for the literal Hartley--Leemans presentation.
# GAP/ACE coset enumeration and GAP Schreier/Tietze/MTC algorithms are required.
if LoadPackage("ace")<>true then Error("ACE required");fi;
ACEIgnoreUnknownDefault:=false;; SizeScreen([100000,24]);;
Need:=function(b,s) if not b then Error(s);fi;end;;
f:=FreeGroup("s0","s1","s2","s3");;a:=GeneratorsOfGroup(f);;
rels:=Concatenation(List(a,x->x^2),[(a[1]*a[2])^5,(a[2]*a[3])^3,
 (a[3]*a[4])^5,Comm(a[1],a[3]),Comm(a[1],a[4]),Comm(a[2],a[4]),
 (a[2]*a[3]*a[4])^5]);;
sub:=[a[3]*a[2],(a[1]*a[2])^2*a[3]*a[4]*a[1],
 a[1]*a[2]*a[1]*a[4]*a[3]*a[4]*a[2]];;
ct:=ACECosetTableFromGensAndRels(a,rels,sub:
 workspace:=5000000,max:=100000,time:=30,silent);;
Need(ct<>fail and Length(ct[1])=20,"complete index twenty");
PrintTo("index20.json",ct,"\n");
w:=f/rels;;rp:=PresentationSubgroupRrs(w,ct,"h");;
primary:=PrimaryGeneratorWords(rp);;
Need(Length(primary)=3,"three actual primary subgroup words");
# Involutory generators permit erasing the signs in the primary words.
Need(List(primary,x->List(LetterRepAssocWord(x),AbsInt))=
 List(sub,LetterRepAssocWord),"literal subgroup words");
PrintTo("schreier_initial.json",List(RelatorsOfFpGroup(FpGroupPresentation(rp)),LetterRepAssocWord),"\n");
TzInitGeneratorImages(rp);;TzGoGo(rp);;
outgens:=GeneratorsOfPresentation(rp);;
Need(Length(outgens)=2,"two final generators");
final:=List(RelatorsOfFpGroup(FpGroupPresentation(rp)),LetterRepAssocWord);;
# FpGroupPresentation renumbers the two surviving generators.
Need(final=[[2,2,2],[1,2,1,-2,1,2,-1,2,1]],"exact two relators");
images:=rp!.imagesOldGens;;
Need(images{[1..3]}=[[-1,-2,1],[-2,-1,2],[-2,-1,2,2,1]],"tracked images of the three source generators");
PrintTo("schreier_final.json",[final,images,rp!.preImagesNewGens],"\n");
# This is a new presented group; no finite permutation quotient enters here.
t:=FreeGroup("x","y");;x:=t.1;;y:=t.2;;
l:=t/[y^3,x*y*x*y^-1*x*y*x^-1*y*x];;
aug:=NEWTC_CosetEnumerator(FreeGeneratorsOfFpGroup(l),RelatorsOfFpGroup(l),
 [l.1],true,false:abelian:=true,limit:=500000,quiet:=true);;
Need(aug<>fail and aug.index=175560,"complete cyclic subgroup index");
Need(Length(aug.subgens)=1,"one specified cyclic generator");
matrix:=NEWTC_AbelianizedRelatorsSubgroup(aug);;
Need(ForAll(matrix,r->Length(r)=1),"one-column relation matrix");
exponents:=List(matrix,r->r[1]);;
Need(Gcd(exponents)=171,"cyclic subgroup relation gcd");
PrintTo("cyclic_relations.json",matrix,"\n");
PrintTo("presentation_summary.json",[20,175560,Gcd(exponents),20*175560*Gcd(exponents),Length(matrix)],"\n");
Print("L095_PRESENTATION_PASS 20 175560 171 600415200\n");
QUIT;
```

Complete reused marking construction

```
# Ambient marking: seed-search algorithm reused from accepted phase40, with reversed order.
# No subgroup or geometry census is an input.
if LoadPackage("tomlib")<>true then Error("TomLib is required");fi;
SizeScreen([100000,24]);;
CheckL084:=function(test,msg) if not test then Error(msg);fi;end;;
G:=UnderlyingGroup(TableOfMarks("J1"));;
CheckL084(Size(G)=175560,"ambient J1 order");;
FindJankoSeed:=function(K)
 local a,b,c,d,iv,bs;
 a:=GeneratorsOfGroup(K)[1];
 CheckL084(Order(a)=2,"ambient first generator");
 iv:=Filtered(Elements(Centralizer(K,a)),x->Order(x)=2);
```

```
 for d in iv do
  if d=a then continue;fi;
  bs:=Filtered(Elements(Centralizer(K,d)),x->Order(x)=2 and Order(a*x)=5);
  for c in iv do
   if Order(c*d)<>5 then continue;fi;
   for b in bs do
    if Order(b*c)=3 and (a*b*c)^5=One(K)
      and ((d*c*b)^5*a)^3=One(K) and Group(a,b,c,d)=K then
     return [a,b,c,d];
    fi;
   od;
  od;
 od;
 Error("no source seed");
end;;
s:=FindJankoSeed(G);;
jmarks:=Reversed(s);;
pmarks:=[(1,2)(3,6)(4,7)(5,10)(8,14)(9,15)(11,19)(12,16)(13,17)(18,20),
(1,3)(2,5)(4,8)(6,13)(7,12)(9,17)(10,15)(11,18)(14,19)(16,20),
(1,3)(2,6)(4,9)(5,11)(7,15)(8,16)(10,19)(12,14)(13,18)(17,20),
(1,4)(2,7)(3,8)(5,12)(6,14)(9,18)(10,16)(11,17)(13,19)(15,20)];;
P:=Group(pmarks);; W:=DirectProduct(G,P);; ej:=Embedding(W,1);; ep:=Embedding(W,2);;
marks:=List([1..4],i->Image(ej,jmarks[i])*Image(ep,pmarks[i]));;
CheckL084(Group(marks)=W and Size(W)=600415200,"full product generation");;
H0:=Group(marks{[2,3,4]});;H3:=Group(marks{[1,2,3]});;om:=(marks[1]*marks[2]*marks[3])^5;;
CheckL084(ForAll(marks,x->Order(x)=2),"four involutions");;
CheckL084(List([1..3],i->Order(marks[i]*marks[i+1]))=[5,3,5],"exact adjacent labels");;
CheckL084(ForAll([[1,3],[1,4],[2,4]],ij->Comm(marks[ij[1]],marks[ij[2]])=One(W)),"all string relations");;
CheckL084((marks[2]*marks[3]*marks[4])^5=One(W),"hemi-icosahedral presentation relator");;
CheckL084(Size(P)=3420 and Size(H0)=60 and Size(H3)=120 and Order(om)=2 and
ForAll(GeneratorsOfGroup(H3),x->Comm(x,om)=One(W)),"exact local groups and central antipode");;
Print("MODEL ",Size(W)," ENDGROUPS ",Size(H0)," ",Size(H3)," OMEGA ",Order(om),"\n");
```

Full marked-geometry interfaces

```
# Actual marked finite models and all intersection identities.
Read("programs/ambient_marking.g");;
Need:=CheckL084;;
IP:=function(r)
 local sets,pars,i,j;
 sets:=Combinations([1..Length(r)]);pars:=List(sets,s->Group(Concatenation([One(r[1])],r{s})));
 for i in [1..Length(sets)] do for j in [1..Length(sets)] do
  Need(Intersection(pars[i],pars[j])=pars[Position(sets,Intersection(sets[i],sets[j]))],"full intersection
  property");
 od;od;
 return Length(sets)^2;
end;;
jIP:=IP(jmarks);;pIP:=IP(pmarks);;wIP:=IP(marks);;
for r in [jmarks,pmarks,marks] do
 Need(ForAll(r,x->Order(x)=2) and List([1..3],i->Order(r[i]*r[i+1]))=[5,3,5],"exact marker and adjacent orders
 in each quotient");
 Need(ForAll([[1,3],[1,4],[2,4]],ij->Comm(r[ij[1]],r[ij[2]])=One(r[1])),"all string commutators in each
 quotient");
 Need((r[2]*r[3]*r[4])^5=One(r[1]),"hemi relation in each quotient");
od;
Need(IsSimpleGroup(G) and IsSimpleGroup(P),"two actual simple factors");
iso:=IsomorphismGroups(P,PSL(2,19));;Need(iso<>fail,"named PSL identification");
jo:=(jmarks[1]*jmarks[2]*jmarks[3])^5;;jn:=jo*jmarks[4];;
po:=(pmarks[1]*pmarks[2]*pmarks[3])^5;;pn:=po*pmarks[4];;
Need(Order(jo)=2 and Order(jn)=3 and Order(po)=1 and Order(pn)=2,"exact quotient normal words");
Need(NormalClosure(G,Group(jo))=G and NormalClosure(P,Group(pn^3))=P,"full normal closures in factors");
Need(List([jmarks,pmarks],r->List([[1,2,3],[2,3,4]],s->Size(Group(r{s}))))=[[120,60],[60,60]],"local orders");
jd:=Group(jmarks{[1,2,3]});;ji:=Group(jmarks{[2,3,4]});;
Need(IsomorphismGroups(jd,DirectProduct(CyclicGroup(2),AlternatingGroup(5)))<>fail and
IsomorphismGroups(ji,AlternatingGroup(5))<>fail,"named endpoint groups");
# Exact source prime-order words, each tested in the endpoint product.
jwords:=[jmarks[2]*jmarks[3],jmarks[1]*jmarks[2],
 jmarks[4]*jmarks[2]*jmarks[3]*jmarks[2]*jmarks[1],
 jmarks[4]*jmarks[3]*jmarks[2]*jmarks[1]*jmarks[2]*jmarks[1],
 jmarks[2]*jmarks[4]*jmarks[3]*jmarks[2]*jmarks[4]*jmarks[1]*jmarks[2]*jmarks[1]*jmarks[3]*jmarks[2]*jmarks[1]
 ];;
jproduct:=Set(Concatenation(List(Elements(ji),a->List(Elements(jd),b->a*b))));;
Need(List(jwords,Order)=[3,5,19,11,7] and ForAll(jwords,x->x in jproduct),"all five literal prime witnesses");
Need(Length(Filtered(ConjugacyClasses(G),c->Order(Representative(c))=2))=1,"unique Janko involution class");
Need(ForAll([3,5,7,11,19],p->Size(SylowSubgroup(G,p))=p),"odd Sylow groups have prime order");
# A second direct finite check: every nonidentity subgroup has an order-p
# subgroup conjugate to one meeting the forbidden endpoint product.
PrimeObstruction:=function(k,r,allowed)
 local product,primes,p,cl,subs,rep,z,hit,out;
 product:=Set(Concatenation(List(Elements(Group(r{[2,3,4]})),a->List(Elements(Group(r{[1,2,3]})),b->a*b))));
 primes:=Set(FactorsInt(Size(k)));out:=[];
 for p in primes do
  subs:=[];
  for cl in ConjugacyClasses(k) do
   z:=Representative(cl);if Order(z)<>p then continue;fi;
   rep:=Group(z);
   if ForAny(subs,h->IsConjugate(k,h,rep)) then continue;fi;
```

```
   Add(subs,rep);
   hit:=First(Difference(product,allowed),b->Order(b)=p and IsConjugate(k,rep,Group(b)));
   Need(hit<>fail,"prime subgroup misses forbidden endpoint product");
  od;
  Add(out,[p,Length(subs)]);
 od;
 return out;
end;;
jobs:=PrimeObstruction(G,jmarks,[One(G),jo]);;
pobs:=PrimeObstruction(P,pmarks,[One(P)]);;
# Reversal of the four exact PSL markers extends to the duality automorphism.
dual:=GroupHomomorphismByImages(P,P,pmarks,Reversed(pmarks));;
Need(dual<>fail and IsBijective(dual),"marked self-duality of the 57-cell");
Need(Size(G)/120=1463 and Size(G)/60=2926 and Size(P)/60=57,"exact source face counts");
Need(Order(om*marks[4])=6,"exact nu order six");
reps:=[[],[1],[2],[4],[1,2],[1,3],[1,4],[2,4],[4,3],[1,2,1],
 [1,2,3],[1,2,4],[1,3,2],[1,3,4],[1,4,3],[2,4,3],[4,3,2],
 [4,3,4],[1,2,1,3],[1,4,3,4]];;
Need(List(reps,w->1^Product(Concatenation([One(P)],pmarks{w})))=[1..20],"all twenty literal coset
representatives");
us:=[marks[3]*marks[2],(marks[1]*marks[2])^2*marks[3]*marks[4]*marks[1],
 marks[1]*marks[2]*marks[1]*marks[4]*marks[3]*marks[4]*marks[2]];;
Need(Size(Group(us))=30020760 and Order(us[2]^-1*us[3]*us[1])=171,"actual index-twenty subgroup and cyclic
image");
PrintTo("geometry_summary.json",[jIP,pIP,wIP,jobs,pobs,1463,2926,57,6],"\n");
Print("L095_GEOMETRY_PASS 768 IP identities; two prime obstructions; marked duality\n");
QUIT;
```